# Python Goto Statement

Python is one of the most popular operating systems in the industry today. Starting from amateurs all the way to professional users, everyone is using Python, but even so, there are some aspects which are yet undiscovered. One such aspect of Python is the native goto statement. So, in this article, we will discuss goto statement in python in the following order:


- What is a Goto Statement?
- Iterations of the Goto Statement
- Computed Goto Statement
- Restrictions


## What is a Goto Statement?

A goto statement can simply be defined as syntax or a piece of code which provides an unconditional jump from the goto statement to one marked as the destination in the contents of the same function. In layman terms, if you want the program to skip a certain number of functions in between you need to use the goto statement.

Note: Although the use of a goto statement is highly probable between most programmers, for auditing purposes it might sometimes be discouraged as tracing the program flow often becomes difficult with the presence of a goto statement. If any situation, the programmer needs to modify the contents of the program and make changes, it becomes difficult to locate the exact destination as the goto statement would have conveniently jumped some portions in the function.

## Syntax

The syntax for the goto statement in Python is as given below.

```
#Syntax-1
goto label;
.
.
label:

#Syntax-2
label:
.
.
goto label;
```

In the above example, the label can be replaced with any text that you require, except the keyword Go and it can be set anywhere in the program, either below or above the go statement as well.

**Quick Fact:** The goto statement was first released on 1st April 2004 as a joke, but programmers across the world took it seriously and started using it.

## Iterations of the Goto Statement

Another code that works the same as a goto statement in Python is comefrom. Both comefrom and the goto statements add flexibility to the overall program in Python, thus allowing one to control program

flow mechanisms and also include accessibility to control flow idioms that were previously out of bounds for them.

In order to use both the goto as well as comefrom statements in Python, one needs to first import them the main library. To do this, type the code mentioned below.

```
from goto import goto, comefrom, label
```

Once the libraries have been imported, you can conveniently use both these functions across your program.

When you use a goto statement in Python, you are basically instructing the interpreter to directly execute another line of code instead of the current one. The target line of code which you want the interpreter to execute at this moment needs to be marked in the section termed “label”. One thing to note about the label tag is the fact that they are mostly random and arbitrary Python identifiers prefixed with a single dot. Example label .myLabel.

## Computed Goto Statement

One of the most common variations of the goto statements used in Python by most programmers is the computed goto statement. In this you mention the python index at the beginning of the code and later refer to it by using a hashtag. Example,

```
x = calculateLabelName()
goto *x
```

Note: The value of x in the above statement should not include the prefix dot as mentioned in the example before this.

## Comefrom

In Python, the `comefrom` statement is basically the opposite of the `goto` statement. In the most simple of terms, its function to the interpreter can be explained via the following statement, “Whenever label X is reached, jump to here instead.”

An example of the `comefrom` statement in action is mentioned below.

```
# ...code 1...

label .somewhere

# ...code 2...

comefrom .somewhere
```

In the above statement, code 2 will not be executed. When the interpreter reaches the line `label .somewhere`, it will directly skip to the next line that is `comefrom .somewhere`.

Another important aspect to note about the `comefrom` statement is the fact that it is mostly always used as a debugging aid in programming. Its use in standalone programming operations is mostly discouraged, as it might sometimes lead to inconvenient and supporting results.

## Restrictions in Goto Statement in Python

Similar to other coding platforms and lines of code, Python too puts a number of restrictions on what both the `goto` as well as the `comefrom` statement can accomplish. Mentioned below are some of the most common restrictions for both the `goto` and `comefrom` statements.

Jumping into the middle of a loop or a `finally` clause is not allowed using either of these statements.

One cannot use either of these statements to jump between functions and or modules.

It cannot be used to jump into an `except` line, because there is no exception line in the first place.

# Example 1: Breaking out from a deeply nested loop:

```
from goto import goto, label
for i in range(1, 10):
    for j in range(1, 20):
        for k in range(1, 30):
            print i, j, k
            if k == 3:
                goto .end
label .end
```

print "Finished"

# Example 2: Cleaning up after something fails:

```
from goto import goto, label

# Imagine that these are real worker functions.
def setUp(): print "setUp"
def doFirstTask(): print 1; return True
def doSecondTask(): print 2; return True
def doThirdTask(): print 3; return False  # This one pretends to fail.
def doFourthTask(): print 4; return True
def cleanUp(): print "cleanUp"

# This prints "setUp, 1, 2, 3, cleanUp" - no "4" because doThirdTask fails.
def bigFunction1():
    setUp()
    if not doFirstTask():
        goto .cleanup
    if not doSecondTask():
        goto .cleanup
    if not doThirdTask():
        goto .cleanup
    if not doFourthTask():
        goto .cleanup

    label .cleanup
    cleanUp()

bigFunction1()
print "bigFunction1 donen"
```

The goto statement is Python is one of the most helpful when it comes to auditing as well as debugging needs. Although it can sometimes be used in day to day programming, using it more than often can sometimes lead to surprising results.